

Web Sémantique

jerome.nobecourt@gmail.com, vaillant@univ-paris13.fr
www-limics.smbh.univ-paris13.fr/Portail

XSD (XML Schema Document)
XSLT (XML Stylesheet Language
Transformations)

Rappel : XML

- XML : eXtensible Markup Language
- permet de stocker n'importe quelle information (semi-)structurée
- le contenu est encapsulé dans des éléments (qui « étiquettent » sa nature)
- les éléments peuvent avoir des attributs (atomiques)
- et ils peuvent être imbriqués

Rappel : XML

```
<enseignants>
  <enseignant sexe="m">
    <prenom>Pascal</prenom>
    <nom>Vaillant</nom>
    <note>Ça c'est <b>moi</b> !</note>
  </enseignant>
  <enseignant sexe="f">
    ...
  </enseignant>
</enseignants>
```

Rappel : XML

élément

<enseignants>

```
<enseignant sexe="m">
  <prenom>Pascal</prenom>
  <nom>Vaillant</nom>
  <note>Ça c'est <b>moi</b> !</note>
</enseignant>
```

```
<enseignant sexe="f">
```

...

```
</enseignant>
```

```
</enseignants>
```

Rappel : XML

```
<enseignants>
```

```
<enseignant sexe="m">
```

*balise ouvrante
de l'élément*

```
<prenom>Pascal</prenom>
```

```
<nom>Vaillant</nom>
```

```
<note>Ça c'est <b>moi</b> !</note>
```

```
</enseignant>
```

```
<enseignant sexe="f">
```

...

```
</enseignant>
```

```
</enseignants>
```

Rappel : XML

```
<enseignants>
```

```
  <enseignant sexe="m">
```

```
    <prenom>Pascal</prenom>
```

```
    <nom>Vaillant</nom>
```

```
    <note>Ça c'est <b>moi</b> !</note>
```

```
  </enseignant>
```

```
  <enseignant sexe="f">
```

```
    ...
```

```
  </enseignant>
```

```
</enseignants>
```

*balise fermante
de l'élément*

Rappel : XML

*élément imbriqué
dans un autre
élément*

```
<enseignants>
  <enseignant sexe="m">
    <prenom>Pascal</prenom>
    <nom>Vaillant</nom>
    <note>Ça c'est <b>moi</b> !</note>
  </enseignant>
  <enseignant sexe="f">
    ...
  </enseignant>
</enseignants>
```

Rappel : XML

```
<enseignants>
  <enseignant sexe="m">
    <prenom>Pascal</prenom>
    <nom>Vaillant</nom>
    <note>Ça c'est <b>moi</b> !</note>
  </enseignant>
  <enseignant sexe="f">
    ...
  </enseignant>
</enseignants>
```

*attribut à valeur
atomique*

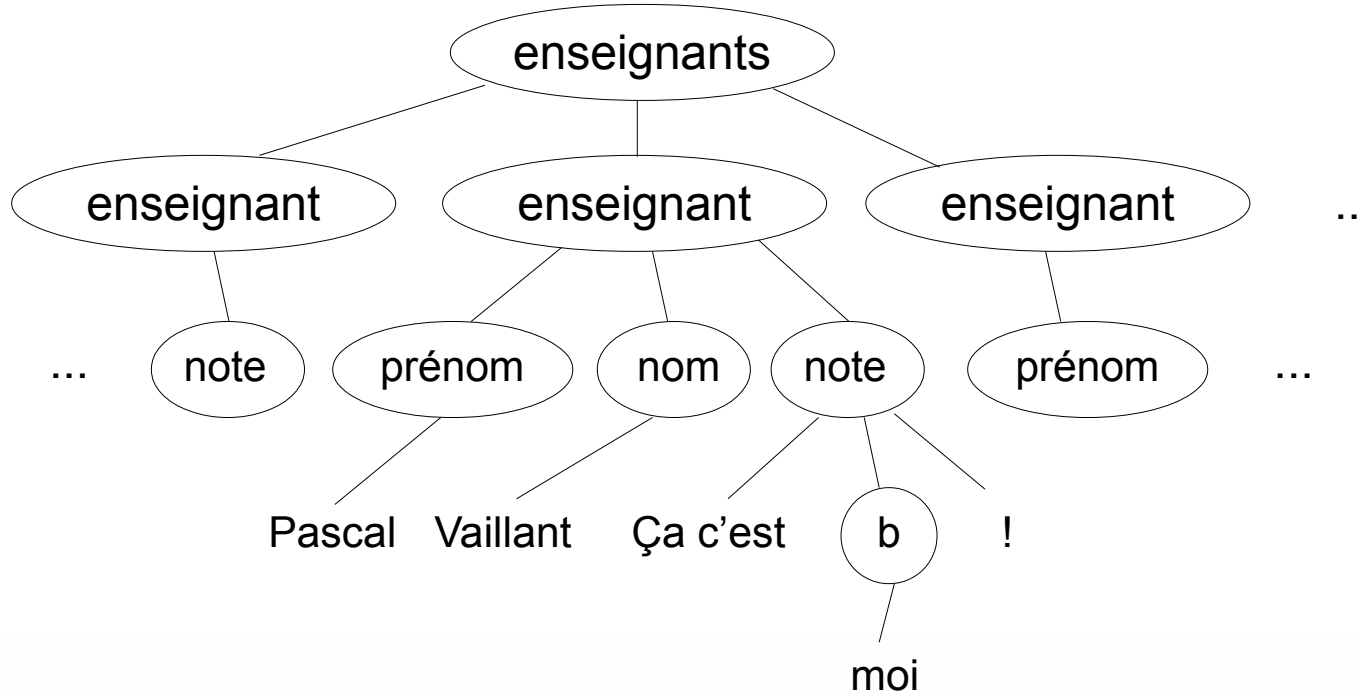
Rappel : XML

```
<enseignants>
  <enseignant sexe="m">
    <prenom>Pascal</prenom>
    <nom>Vaillant</nom>
    <note>Ça c'est <b>moi</b> !</note>
  </enseignant>
  <enseignant sexe="f">
    ...
  </enseignant>
</enseignants>
```

contenu mixte

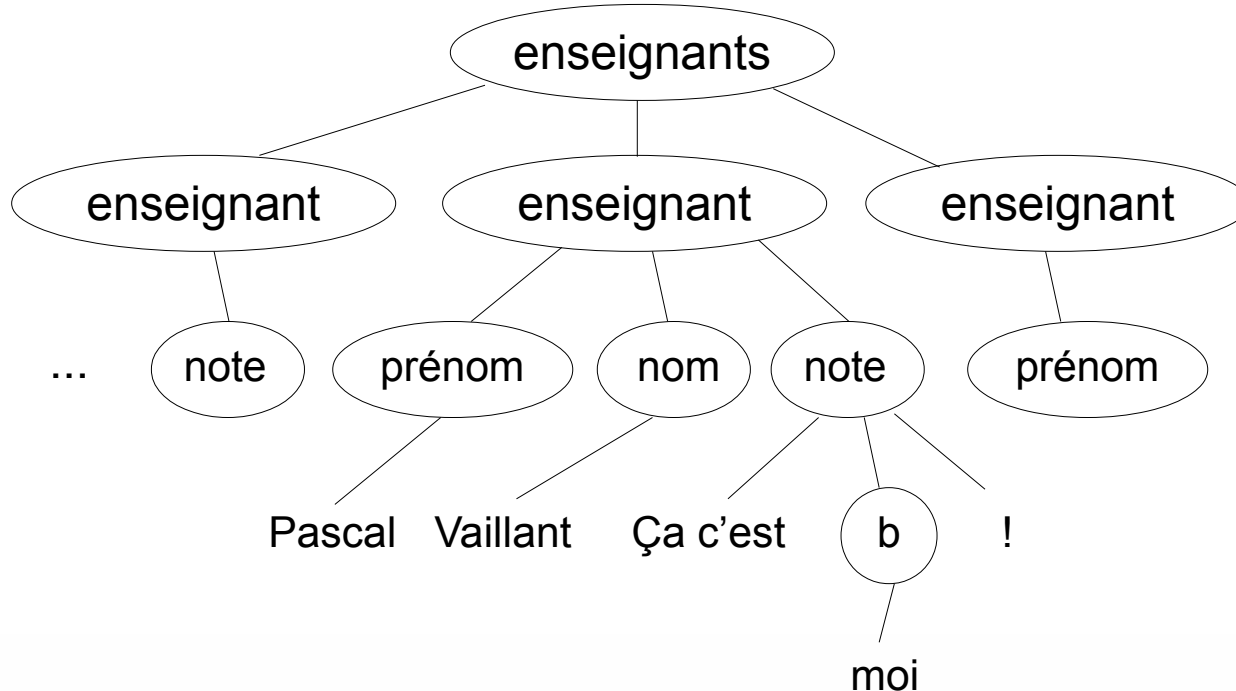
Rappel : XML – *bonne formation*

- Tout document XML est un *arbre*



Rappel : XML – *bonne formation*

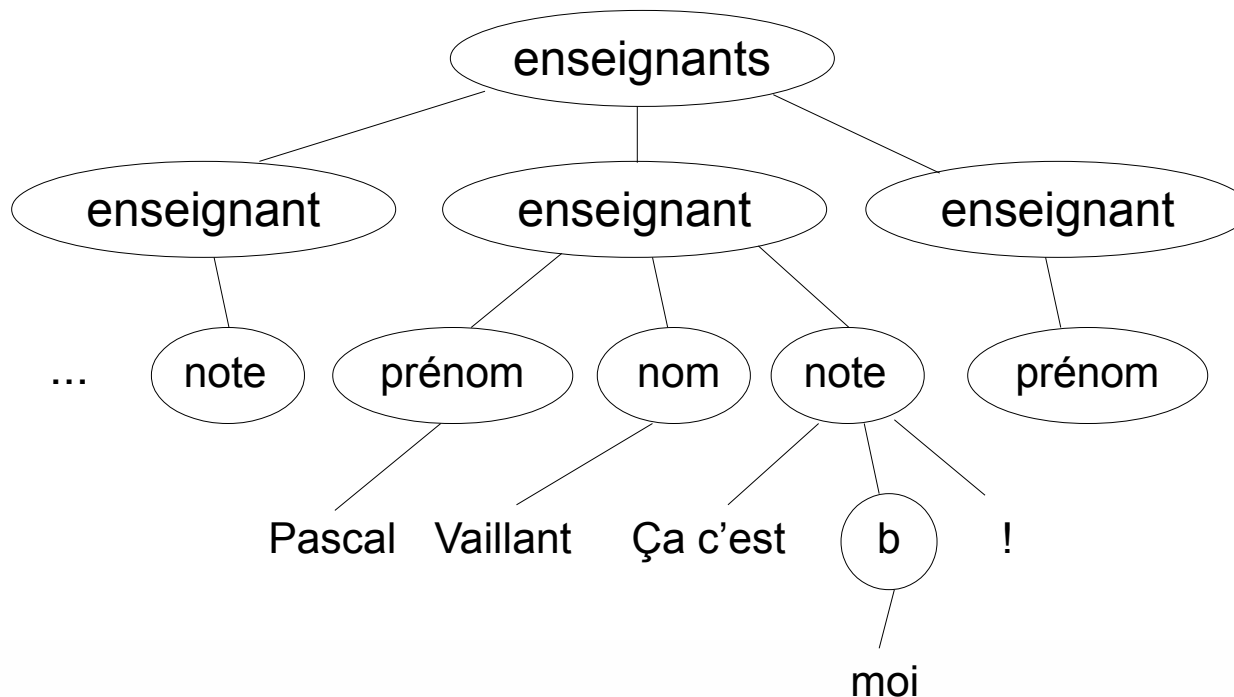
- Tout document XML est un *arbre*



⇒ 1. Tout document a un et un seul nœud racine !

Rappel : XML – *bonne formation*

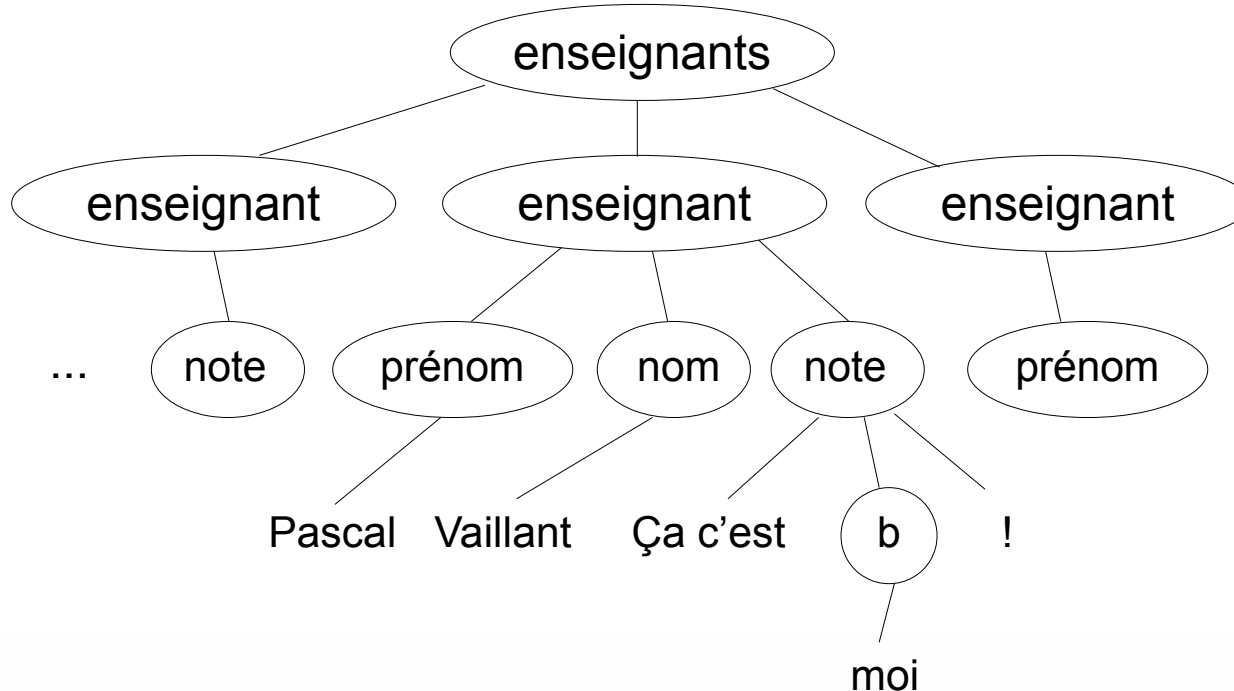
- Tout document XML est un *arbre*



<enseignants>
...
</enseignants>

Rappel : XML – *bonne formation*

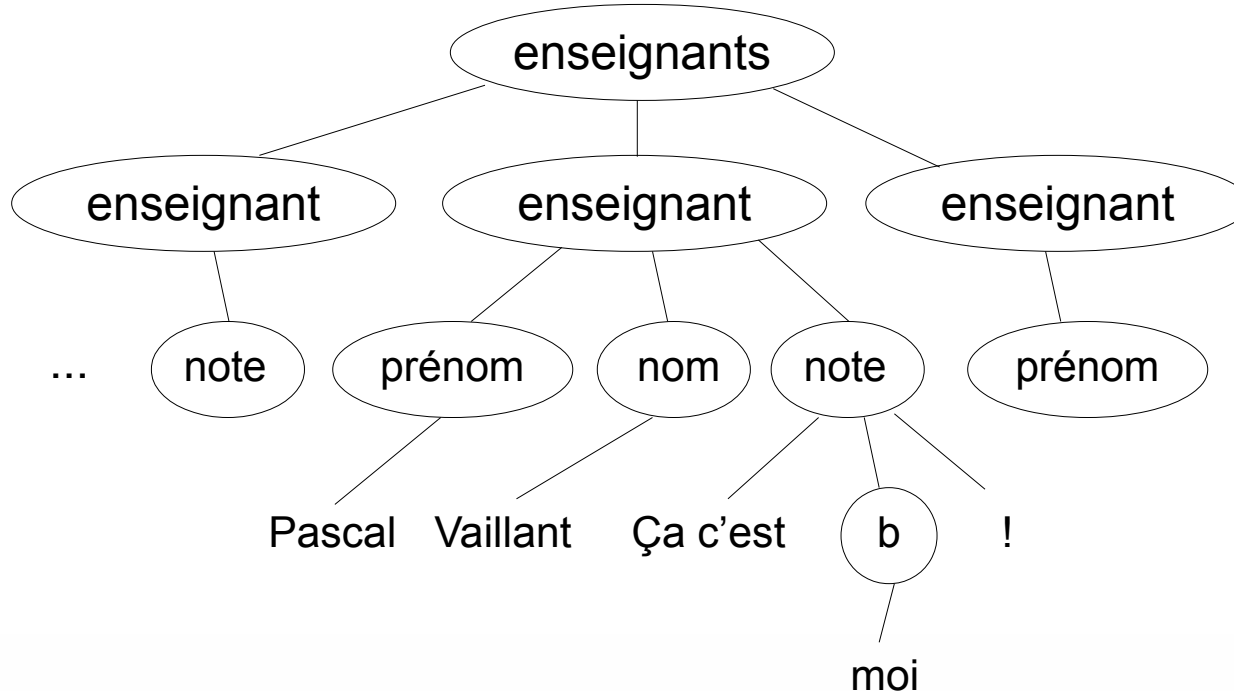
- Tout document XML est un *arbre*



⇒ 2. Toute balise ouverte doit être refermée !

Rappel : XML – *bonne formation*

- Tout document XML est un *arbre*

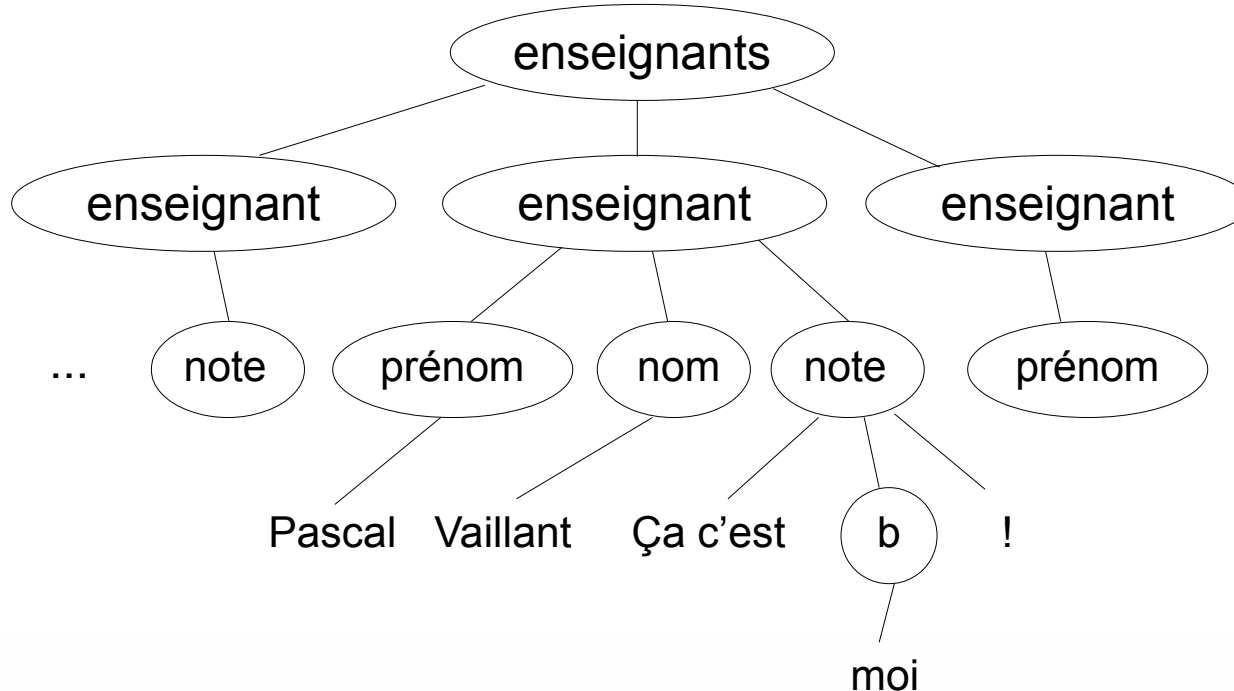


```

<enseignants>
  <enseignant>
    ...
  </enseignant>
  <enseignant>
    ...
  </enseignant>
</enseignants>
  
```

Rappel : XML – *bonne formation*

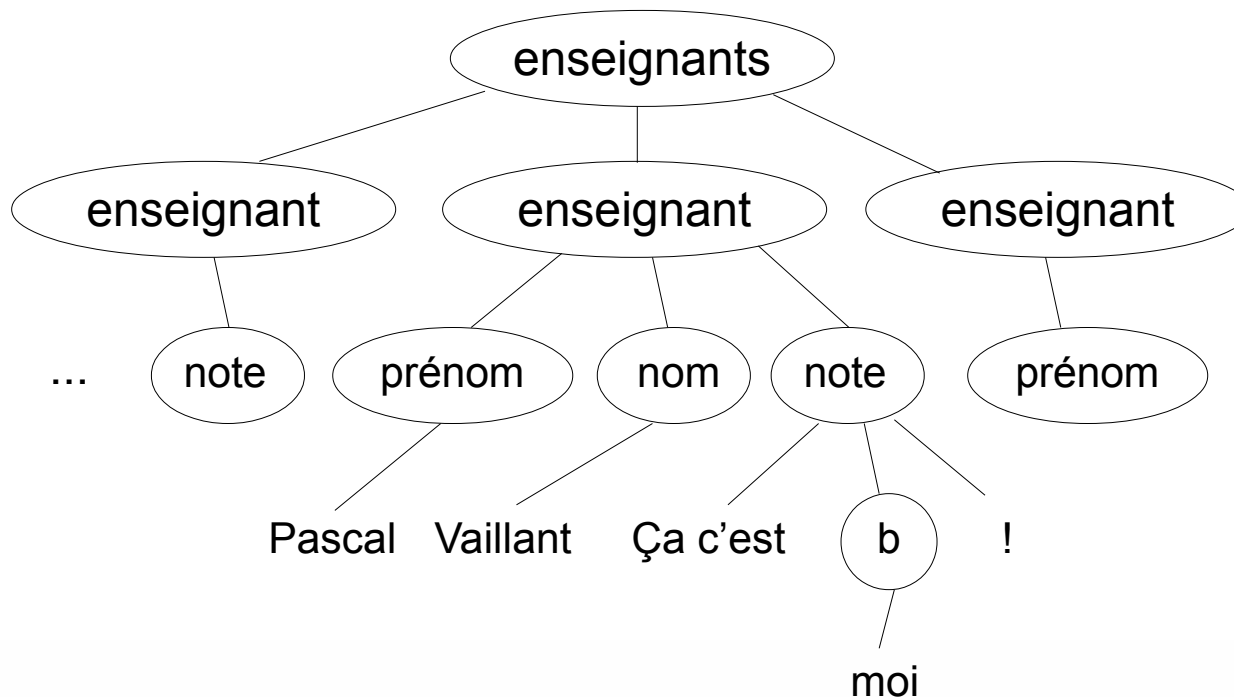
- Tout document XML est un *arbre*



⇒ 3. On ne peut pas refermer un nœud père avant son nœud fils !

Rappel : XML – *bonne formation*

- Tout document XML est un *arbre*



```

<enseignants>
  <enseignant>
    ...
  </note>
  ...
</note>
</enseignant>
<enseignant>
  ...
</enseignant>
</enseignants>
  
```


Rappel : XML – *validité*

- Quand on dit qu'un document XML est bien formé, cela veut simplement dire qu'il s'agit bien de la représentation linéaire d'un *arbre*
- Par ailleurs, un document peut être bien formé mais n'avoir aucun « sens », s'il n'est pas conforme à la représentation de l'information qu'on attend

Rappel : XML – *validité*

- Exemple : si je lis un document dans lequel je m'attends à trouver une liste d'enseignants ...

Rappel : XML – *validité*

- Exemple : si je lis un document dans lequel je m'attends à trouver une liste d'enseignants ...

```
<enseignants>
  <enseignant quantite="3">
    <prenom>Pascal</prenom>
    <nom>Vaillant</nom>
    <prix unite="EUR">49.95</prix>
  </enseignant>
  <article sexe="2070551946">
    <date_peremption>2020-02-18</date_peremption>
  </article>
</enseignants>
```

Rappel : XML – *validité*

- C'est techniquement « bien formé »,
mais ça n'a pas de sens (cherchez l'erreur)

```
<enseignants>
  <enseignant quantite="3">
    <prenom>Pascal</prenom>
    <nom>Vaillant</nom>
    <prix unite="EUR">49.95</prix>
  </enseignant>
  <article sexe="2070551946">
    <date_peremption>2020-02-18</date_peremption>
  </article>
</enseignants>
```

Rappel : XML – *validité*

- Un document XML doit (ou du moins : devrait toujours) se conformer à un type qui définit les variations possibles dans la limite d'une structure
- Le type de documents définit une *syntaxe* (c'est-à-dire une grammaire)
- Celle-ci, implicitement, possède une *sémantique* (c'est-à-dire que les éléments permettent de donner un sens à l'information échangée)

Rappel : XML – *validité*

- En réalité, XML n'est pas un langage, c'est un *méta-langage* ...
- c-à-d un langage qui permet de définir des langages
- **Il n'y a pas de document « XML » sans plus de précision**, il n'y a que des documents balisés dans des **langages spécifiques** qui respectent le formalisme générique XML (XHTML, docbook, OpenDocument, SVG ...)

Rappel : XML – *validité*

- Un document XML est *valide* s'il fait référence à un type de documents bien défini, et s'il est conforme à ce type
- Ex. pour un document qui doit transmettre une liste d'enseignants, l'exemple du transparent 20 n'est pas *valide* (bien qu'il soit *bien formé*)

Rappel : XML – *type de document*

- Il existe deux méthodes pour définir un type de documents :

1. la DTD (*Document Type Definition*), syntaxe spécifique héritée de SGML :

```
<!ELEMENT enseignants (enseignant+)>  
<!ELEMENT enseignant (prenom, nom, note)>  
<!ATTLIST enseignant sexe CDATA #REQUIRED>  
<!ELEMENT prenom (#PCDATA)>  
<!ELEMENT nom (#PCDATA)>  
<!ELEMENT note ANY>
```


Rappel : XML – *type de document*

- Il existe deux méthodes pour définir un type de documents :

2. le XML Schema, qui est lui-même exprimé en XML :

```
<xs:complexType name="type_enseignants">
  <xs:sequence>
    <xs:element name="enseignant" type="type_enseignant"
minOccurs="1" maxOccurs="unbounded"/>
  </xs:sequence>
</xs:complexType>
<xs:element name="enseignants" type="type_enseignants"/>
```

...

Rappel : XML

- On part du principe, dans la suite de ce cours, que l'on manipule des documents XML ...
 - bien formés
 - valides (par rapport à un type de documents identifié)
 - syntaxiquement corrects (en-tête XML présent, codage des caractères cohérent)

Rappel : XML

déclaration XML

```
<?xml version="1.0" encoding="UTF-8"?>  
<!DOCTYPE enseignants SYSTEM "enseignement.dtd">
```

```
<enseignants>  
  <enseignant sexe="m">  
    <prenom>Pascal</prenom>  
    <nom>Vaillant</nom>  
    <note>Ça c'est <b>moi</b> !</note>  
  </enseignant>  
  ...  
</enseignants>
```

Rappel : XML

référence à une DTD

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<!DOCTYPE enseignants SYSTEM "enseignement.dtd">
```

```
<enseignants>
```

```
  <enseignant sexe="m">
```

```
    <prenom>Pascal</prenom>
```

```
    <nom>Vaillant</nom>
```

```
    <note>Ça c'est <b>moi</b> !</note>
```

```
  </enseignant>
```

```
  ...
```

```
</enseignants>
```

Rappel : XML

référence à un schéma

```
<?xml version="1.0" encoding="UTF-8"?>
```

```
<enseignants
```

```
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:noNamespaceSchemaLocation="http://
  www.enseignement.net/xs/enseignement.xsd">
```

```
  <enseignant sexe="m">
```

```
    <prenom>Pascal</prenom>
```

```
    <nom>Vaillant</nom>
```

```
    <note>Ça c'est <b>moi</b> !</note>
```

```
  </enseignant>
```

```
  ...
```

```
</enseignants>
```

Rappel : XML vs. BD relationnelles

- Au fait : se rappeler pourquoi on utilise XML (par comparaison avec des schémas relationnels)
- Approche centrée **document** :
 1. l'*ordre* des éléments compte
 2. le contenu n'est *pas* forcément *atomique*
 3. et (notamment), les éléments peuvent être imbriqués les uns dans les autres *récurivement*



Rappel : XML vs. BD relationnelles

- Au fait : se rappeler pourquoi on utilise XML (par comparaison avec des schémas relationnels)
- Approche centrée **document**
- Si l'on n'a pas ces contraintes (linéarité du document, récursivité de la grammaire), alors XML n'est qu'un « format d'export » possible d'un schéma relationnel
- ... et ce n'est pas le mode de stockage le plus efficace ! (absence d'index)



Rappel : XML vs. BD relationnelles

```
<enseignants>
  <enseignant sexe="m">
    <prenom>Pascal</prenom>
    <nom>Vaillant</nom>
  </enseignant>
  <enseignant sexe="f">
    <prenom>Céline</prenom>
    <nom>Rouveirol</nom>
  </enseignant>
  . . .
</enseignants>
```




Rappel : XML vs. BD relationnelles

```
<enseignants>
  <enseignant sexe="m">
    <prenom>Pascal</prenom>
    <nom>Vaillant</nom>
  </enseignant>
  <enseignant sexe="f">
    <prenom>Céline</prenom>
    <nom>Rouveirol</nom>
  </enseignant>
  . . .
</enseignants>
```

*Ces données
pourraient très bien
être stockées dans
une table*



Rappel : XML vs. BD relationnelles

```
<enseignants>
```

```
  <enseignant sexe="m">
```

```
    <prenom>Pascal</prenom>
```

```
    <nom>Vaillant</nom>
```

```
  </enseignant>
```

```
  <enseignant sexe="f">
```

```
    <prenom>Céline</prenom>
```

```
    <nom>Rouveirol</nom>
```

```
  </enseignant>
```

```
...
```

```
</enseignants>
```

*Ces données
pourraient très bien
être stockées dans
une table*

```
CREATE TABLE enseignants  
(sexe ENUM("m", "f"),  
  prenom VARCHAR(32),  
  nom VARCHAR(32)) ;
```

Rappel : XML vs. BD relationnelles

```
<article>
  <section id="intro">
    <p>La tempête Hagupit a déjà fait plus d'une
vingtaine de victimes aux Philippines.</p>
    <p id="xyz">D'après la Croix-Rouge, au moins vingt
et une personnes ont été tuées à
Borongan<note><p>Borongan&#xA0;: capitale de la région
du Samar oriental.</p></note>, pour la plupart noyées
par la montée des eaux.
  </p>
  ...
</section>
...
</article>
```

Rappel : XML vs. BD relationnelles

```
<article>
```

```
  <section id="intro">
```

```
    <p>La tempête Hagupit a déjà fait plus d'une
vingtaine de victimes aux Philippines.</p>
```

```
    <p id="xyz">D'après la Croix-Rouge, au moins vingt
et une personnes ont été tuées à
Borongan<note><p>Borongan&#xA0;: capitale de la région
du Samar oriental.</p></note>, pour la plupart noyées
par la montée des eaux.
```

```
  </p>
```

```
  ...
```

```
</section>
```

```
...
```

```
</article>
```

Ces données constituent un document, pas une table !



Rappel : XML vs. BD relationnelles

```
<article>
```

```
  <section id="intro">
```

```
    <p>La tempête Hagupit a déjà fait plus d'une  
vingtaine de victimes aux Philippines.</p>
```

```
    <p id="xyz">D'après la Croix-Rouge, au moins vingt  
et une personnes ont été tuées à  
Borongan<note><p>Borongan&#xA0;: capitale de la région  
du Samar oriental.</p></note>, pour la plupart noyées  
par la montée des eaux.
```

```
  </p>
```

```
  ...
```

```
</section>
```

```
...
```

```
</article>
```

L'ordre compte.

*Ces données constituent un
document, pas une table !*

Rappel : XML vs. BD relationnelles

```
<article>
```

```
  <section id="intro">
```

```
    <p>La tempête Hagupit a déjà fait plus d'une
vingtaine de victimes aux Philippines.</p>
```

```
    <p id="xyz">D'après la Croix-Rouge, au moins vingt
et une personnes ont été tuées à
Borongan<note><p>Borongan&#xA0;: capitale de la région
du Samar oriental.</p></note>, pour la plupart noyées
par la montée des eaux.
```

```
  </p>
```

```
  ...
```

```
</section>
```

```
...
```

```
</article>
```

Ces données constituent un document, pas une table !

L'ordre compte.

On trouve des <p> dans des <note> dans des <p>

XML Schema

- En 2004 le W3C (World Wide Web Consortium) a proposé une norme permettant de définir un type de document XML dans une syntaxe qui est elle-même du XML.
→ XML Schema.

XML Schema

- Le langage DTD a l'avantage de la simplicité.
- Il permet d'exprimer la structure du document sous forme de langage régulier.
- Mais il a plusieurs inconvénients :
 - Pas de typage des données élémentaires.
 - Pas de modularité : pas de séparation élément/type
 - Pas d'expression possible de contraintes internes
 - Pas d'extension possible

XML Schema

- Le schéma XML est lui-même un format de documents XML.
- Avantages du format XML :
 - Propre
 - Ré-utilisable
 - Outils standard d'édition, de validation, d'échange
- Inconvénient : un peu plus lourd à charger et à valider

XML Schema

- Tout schéma XML fait référence à l'espace de nommage de XML Schema :

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">
```
- Par convention, le préfixe utilisé est souvent xs ou xsd.
- Les éléments de définition sont tous précédés de ce préfixe.

XML Schema

- Dans une DTD, on ne peut définir que des expressions régulières bâties sur deux types simples rudimentaires : CDATA et PCDATA
- Dans un schéma XML, on peut définir un ensemble organisé de types de données spécifiques (sur le modèle « orienté objet »)
- On peut adapter finement la description des données à un besoin spécifique

XML Schema

- Un ensemble assez riche de types prédéfinis :
- *Types simples prédéfinis* :
 - Numériques
 - Chaînes
 - Temporels

XML Schema

- Types prédéfinis numériques :
- booléen, décimal, entier, entier signé ou non signé, pouvant ou non inclure zéro, positif ou négatif, codé sur 8, 16, 32 ou 64 bits ...
- nombres « à virgule flottante » codés selon la norme IEEE 754 (*float, double*)

XML Schema

- Types prédéfinis « chaînes » :
 - string : chaîne simple (sans restriction)
 - normalizedString : chaîne normalisée (pas de sauts de ligne ou de tabulations)
 - token : chaîne normalisée sans espaces inutiles
 - name : « nom » XML
 - ID, IDREF : identifiants XML

XML Schema

- Types dérivés :
- Il existe deux sortes de types dérivés :
 - les types simples, définis par extension ou restriction de types prédéfinis
 - les types « complexes », qui permettent de définir la structure interne des éléments du type.

XML Schema

- Types simples :
- Définis par restriction (ex. : chaîne de caractères ne pouvant prendre sa valeur que dans un ensemble fini de valeurs énumérées)
- Définis par extension (ex. : définir un type d'élément contenant une chaîne, et ayant en plus un attribut).

XML Schema

- Combinaisons ensemblistes de types simples :
- On peut en outre former un nouveau type simple en le définissant à partir d'autres types comme :
 - une liste
 - une union

d'autres éléments de types simples

XML Schema

- Types complexes :
- Les types complexes peuvent être définis à partir des types simples pour former des éléments ayant une structuration interne :
 - contenu mixte
 - conjonction ou disjonction d'autres types

XML Schema

- Déclaration de contenu mixte :
 - attribut **mixed="true"**
- Opérateurs de construction de types complexes :
 - séquence ordonnée (**xsd:sequence**)
 - ensemble non-ordonné (**xsd:all**)
 - disjonction (choix d'un élément parmi plusieurs) (**xsd:choice**)

XML Schema

- Exemples d'utilisation (1) :

```
<xs:element name="livre">
  <xs:complexType>
    <xs:all>
      <xs:element name="titre" type="xs:string"
minOccurs="1" maxOccurs="1"/>
      <xs:element name="auteur" type="type_auteur"
minOccurs="1" maxOccurs="1"/>
      <xs:element name="date" type="xs:date"
minOccurs="1" maxOccurs="1"/>
    </xs:all>
  </xs:complexType>
</xs:element>
```

XML Schema

- Exemples d'utilisation (2) :

```
<xs:complexType name="type_auteur">
  <xs:sequence>
    <xs:element name="prenom" type="xs:string"
minOccurs="1" maxOccurs="1"/>
    <xs:element name="nom" type="xs:string"
minOccurs="1" maxOccurs="1"/>
    <xs:element name="pseudonyme" type="xs:string"
minOccurs="0" maxOccurs="unbounded"/>
  </xs:sequence>
</xs:complexType>
```

```
<xs:element name="enseignant" type="type_enseignant"/>
```

XML Schema

- Exemples d'utilisation (3) :

```
<xs:complexType name="type_evenement">
  <xs:choice>
    <xs:element name="prise_parole" type="xs:string"/>
    <xs:element name="indication" type="type_indic"/>
  </xs:choice>
</xs:complexType>

<xs:element name="evenement" type="type_evenement"/>
```

XML Schema

- Autre possibilité offerte avec XML Schema (et pas avec les DTD) : créer des contraintes de clés étrangères
- On peut déclarer que le contenu d'un nœud situé dans une certaine partie de l'arborescence doit forcément être retrouvé dans un nœud situé dans une autre partie de l'arborescence
(pour que le document soit valide)

```
<?xml version="1.0" encoding="UTF-8"?>
<corpus>
  <en_tete_corpus>
    <titre_complet>Extrait de l'émission télévisée hebdomadaire, An nou kozé divini yich nou
    Date: 10 janvier 2006 de 20h10 à 20h55</titre_complet>
    <editeur>Pascal Vaillant</editeur>
    <langues><langue lang="acf"/><langue lang="fra"/><langue lang="mul"/></langues>
    <locuteurs>
      <locuteur><nom_abrege>A</nom_abrege></locuteur><locuteur><nom_abrege>B</nom_abrege></locuteur>
      <locuteur><nom_abrege>C</nom_abrege></locuteur><locuteur><nom_abrege>D</nom_abrege></locuteur>
    </locuteurs>
  </en_tete_corpus>
  <texte>
    <date_recueil>2006-01-10</date_recueil>
    <titre_texte lang="fra">Sujet: L'école et la famille: stratégies de réussite scolaire</titre_texte>
    <prise_de_parole id="e1">
      <identification_du_locuteur>A</identification_du_locuteur>
      <texte_ligne lang="acf">
        <segment lang="fra">Alors c'est vrai que le plus souvent</segment> nou ka wè, an didan lé
        zorientasion lé jenn manmay ka chwasi an branch épi <segment lang="fra">enfin de compte</segment>, bon, soit
        i ni plas yo adan é yo ka kontinié, soit yo chwasi an branch pa ni plas yo ka mété yo adan an lot. Épi
        <segment lang="fra">le plus souvent</segment> soit nou ka trouvé dé jenn mamay ki a l'aise épi nou ka trouvé
        osi dé jenn mamay ki ka sa yo ka di an bon fwansé <segment lang="fra">en échec scolaire</segment>.
      </texte_ligne>
    </prise_de_parole>
  </texte>
  (...)

```


- Dans le Xschema :

```
<xs:element name="corpus">
  <xs:complexType>
    <xs:sequence>
      <xs:element name="en_tete_corpus" type="type_en_tete_corpus"/>
      <xs:element name="texte" type="type_texte_corpus" minOccurs="1" maxOccurs="unbounded"/>
    </xs:sequence>
  </xs:complexType>
  <xs:key name="id_locuteur">
    <xs:selector xpath="./en_tete_corpus/locuteurs/locuteur"/>
    <xs:field xpath="nom_abrege"/>
  </xs:key>
  <xs:keyref name="locuteur_prise_de_parole" refer="id_locuteur">
    <xs:selector xpath="//prise_de_parole"/>
    <xs:field xpath="identification_du_locuteur"/>
  </xs:keyref>
  <xs:key name="code_langue">
    <xs:selector xpath="./en_tete_corpus/langues/langue"/>
    <xs:field xpath="@lang"/>
  </xs:key>
  <xs:keyref name="langue_ligne" refer="code_langue">
    <xs:selector xpath="//prise_de_parole/ligne/texte_ligne"/>
    <xs:field xpath="@lang"/>
  </xs:keyref>
  (...)
```

XML Schema

- Tutoriel de référence avec exemples :

<https://www.w3.org/TR/xmlschema-0/>

- Spécifications formelles :

- Structures et constructeurs :

<https://www.w3.org/TR/xmlschema-1/>

- Types de données :

<https://www.w3.org/TR/xmlschema-2/>

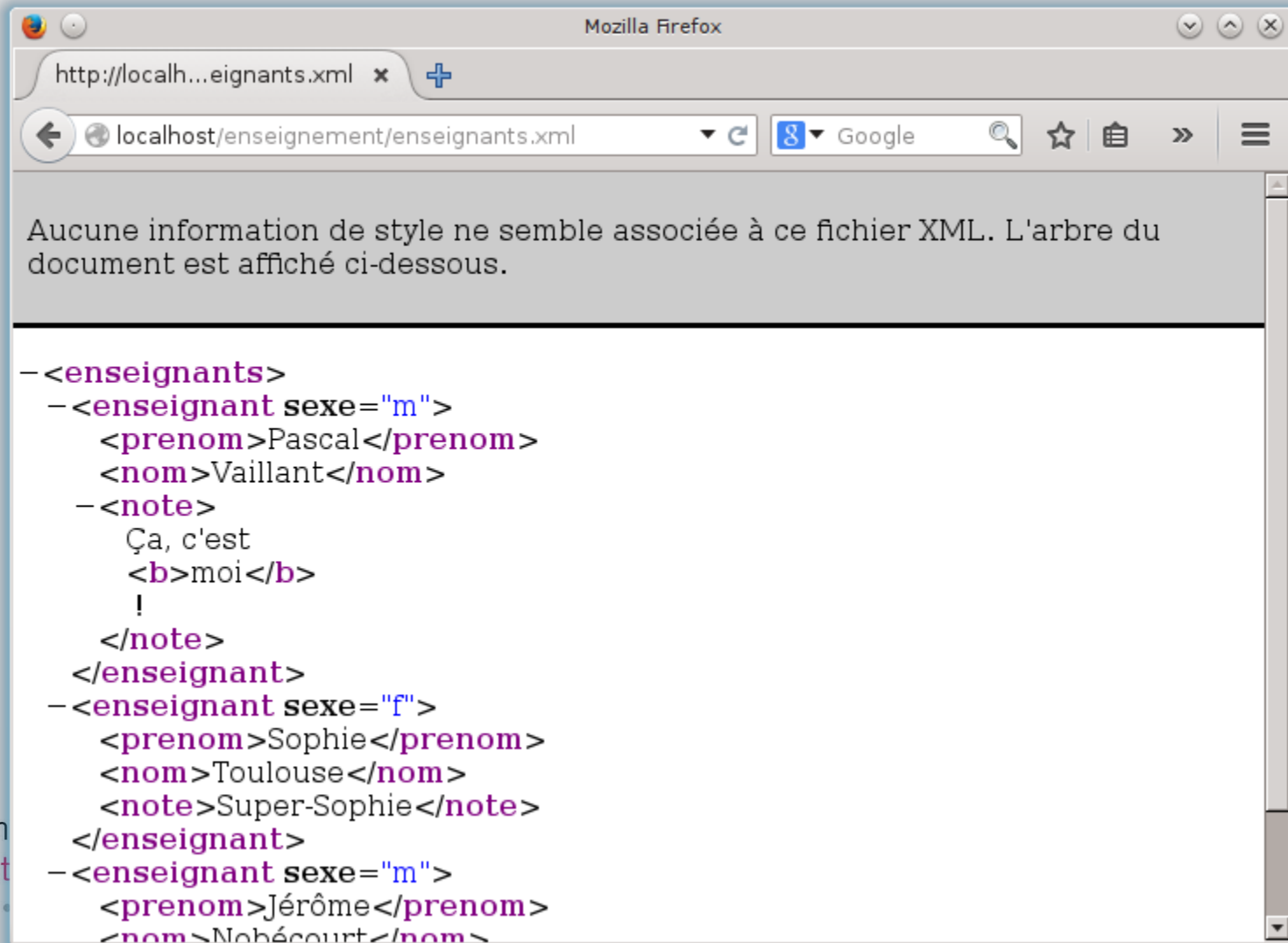
Visualisation de XML

- XML est fait pour définir des données qui ont un sens, pas spécifiquement pour être un format d'affichage.
- HTML est fait pour être un format d'affichage (et d'ailleurs, on peut le spécifier en XML : c'est XHTML) ;
- Mais un type de documents XML, *dans le cas général*, contient des descriptions de données étiquetées, pas des instructions de formatage.

Visualisation de XML

- Que se passe-t-il quand on ouvre un fichier XML dans un navigateur ?

Visualisation de XML

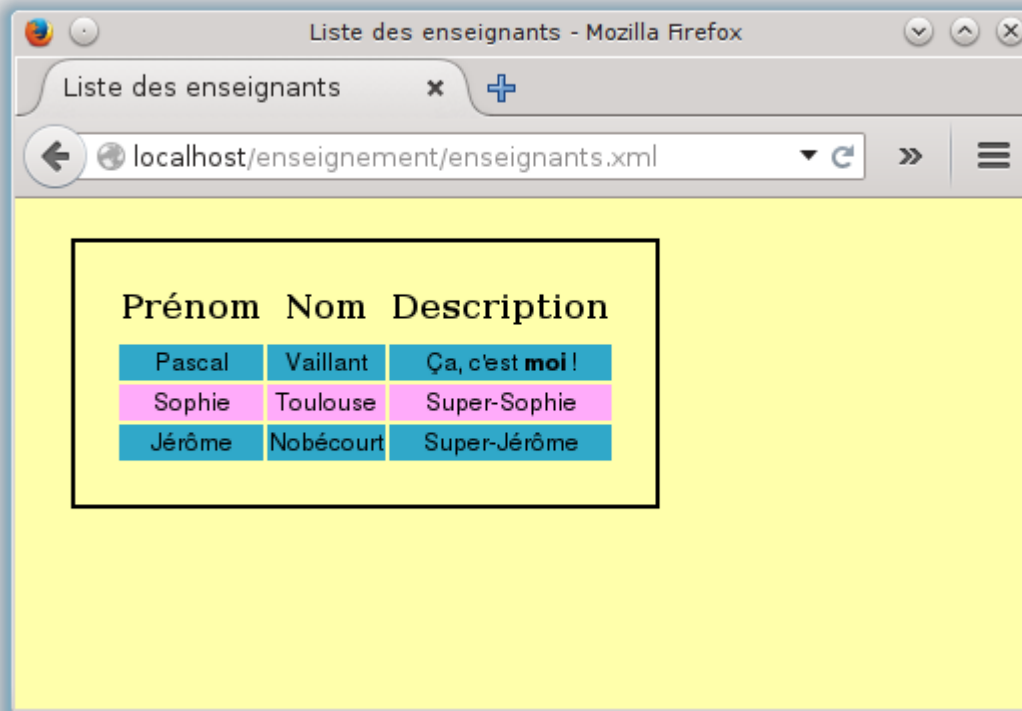


Visualisation de XML

- Que se passe-t-il quand on ouvre un fichier XML dans un navigateur ?
- C'est laid.

Visualisation de XML

- Ce qu'on aimerait voir, c'est ça :



Visualisation de XML

- C'est-à-dire ce qu'on obtiendrait avec du code HTML :

```
<html lang="fr" dir="ltr">
<head><title>Liste des enseignants</title>
<link rel="stylesheet" type="text/css"
  href="enseignement.css" media="screen"></head>
<body><table class="enseignants">
<tr class="titres">
<td>PrÃ©nom</td><td>Nom</td><td>Description</td></tr>
<tr class="masc">
<td>Pascal</td><td>Vaillant</td><td><td>Ãa, c'est <b>moi</b></td></td></tr>
<tr class="femn">
<td>Sophie</td><td>Toulouse</td><td><note>Super - Sophie</note></td></tr>
...

```


Visualisation de XML

- Mais on aimerait conserver l'avantage d'avoir les informations balisées en XML ...
- parce que le HTML n'apporte pas d'information sur la structure logique des données

Visualisation de XML

- Mais on aimerait conserver l'avantage d'avoir les informations balisées en XML ...
- parce que le HTML n'apporte pas d'information sur la structure logique des données
- Solution :

XSLT (XML Stylesheet Language Transformation)

XSLT

- XSLT transforme ceci :

```
<?xml version="1.0"?>
<enseignants>
  <enseignant sexe="m">
    <prenom>Pascal</prenom>
    <nom>Vaillant</nom>
    <note>Ça c'est
<b>moi</b> !</note>
  </enseignant>
  <enseignant sexe="f">
    <prenom>Sophie</prenom>
    <nom>Toulouse</nom>
    <note>Super-Sophie</note>
  </enseignant>
  ...
</enseignants>
```

67

XSLT

- XSLT transforme ceci :

```
<?xml version="1.0"?>
<enseignants>
  <enseignant sexe="m">
    <prenom>Pascal</prenom>
    <nom>Vaillant</nom>
    <note>Ça c'est
<b>moi</b> !</note>
  </enseignant>
  <enseignant sexe="f">
    <prenom>Sophie</prenom>
    <nom>Toulouse</nom>
    <note>Super-Sophie</note>
  </enseignant>
  ...
</enseignants>
```

```
<html lang="fr" dir="ltr">
<head><title>Liste des
enseignants</title>
<link rel="stylesheet" type="text/css"
href="enseignement.css"
media="screen"></head>
<body><table class="enseignants">
<tr class="titres">
<td>PrÃ©nom</td><td>Nom</
td><td>Description</td></tr>
<tr class="masc">
<td>Pascal</td><td>Vaillant</
td><td><td>Ãa, c'est
<b>moi</b>Â !</td></tr>
<tr class="femn">
<td>Sophie</td><td>Toulouse</
td><td><td>Super-Sophie</td></tr>
...
</table>
```

XSLT

- XSLT transforme du XML en HTML ...

69

www.univ-paris13.fr

www.iut-bobigny.univ-paris13.fr

Villetaneuse • Bobigny • Saint-Denis • Saint-Denis - La Plaine • Argenteuil

www.limics.smbh.univ-paris13.fr/Portail

Jerome.nobecourt@gmail.com

vaillant@univ-paris13.fr



L'Université Paris 13 est
membre fondateur de :



XSLT

- XSLT transforme du XML en HTML ... ?

70

www.univ-paris13.fr

www.iut-bobigny.univ-paris13.fr

Villetaneuse • Bobigny • Saint-Denis • Saint-Denis - La Plaine • Argenteuil

www.limics.smbh.univ-paris13.fr/Portail

Jerome.nobecourt@gmail.com

vaillant@univ-paris13.fr



L'Université Paris 13 est
membre fondateur de :



XSLT

- XSLT transforme du XML en HTML ...
- HTML est une « sorte » de XML

71

XSLT

- XSLT transforme du XML en HTML ...
- HTML est une « sorte » de XML
- Donc XSLT transforme du XML en XML

72

XSLT

- XSLT transforme n'importe quelle « sorte » de XML (n'importe quel *type de document* défini en XML) en n'importe quelle autre « sorte » de XML (le plus souvent : en HTML)
- XSLT permet de définir une **grammaire de transformation d'arbres**

73

Qu'est-ce qu'une « feuille » XSLT ?

- Une « feuille de style » XSLT est un document qui définit quelles règles doivent permettre de passer du document source au document transformé. Ex :
- un élément `<enseignants>` devient un élément `<html>` qui contient un `<body>` qui contient une `<table>` ...
- un élément `<enseignant>` devient une ligne dans la table
- un élément `<nom>` devient une colonne dans la ligne ...

Qu'est-ce que XSLT ?

- Le langage XSLT est lui-même défini comme un type de document XML ! Une « feuille de style » XSLT est donc, à la base, un document XML.
- XSLT est un type de documents XML qui est défini par une spécification du W3C (World Wide Web Consortium) :

<http://www.w3c.org/TR/xslt20/>

75

Qu'est-ce que XSLT ?

- Une feuille XSLT 2.0 est ainsi un document XML
 - bien formé (bien sûr) ;
 - valide conformément au schéma de documents qui implémente la spécification du W3C :
<http://www.w3.org/2007/schema-for-xslt20.xsd>
 - déclaré relativement à l'*espace de nommage xsl* :
<http://www.w3.org/1999/XSL/Transform>

Quand intervient XSLT ?

- Lorsqu'un programme client (typiquement, un navigateur) doit interpréter et transposer un document XML :
 - il regarde si une feuille de style XSLT est indiquée ;
 - si ce n'est pas le cas, il affiche une représentation brute de l'arbre du document XML (DOM XML) ;
 - si c'est le cas, il l'utilise pour transformer le flux XML d'entrée en un autre format (typiquement, HTML)

Quand intervient XSLT ?

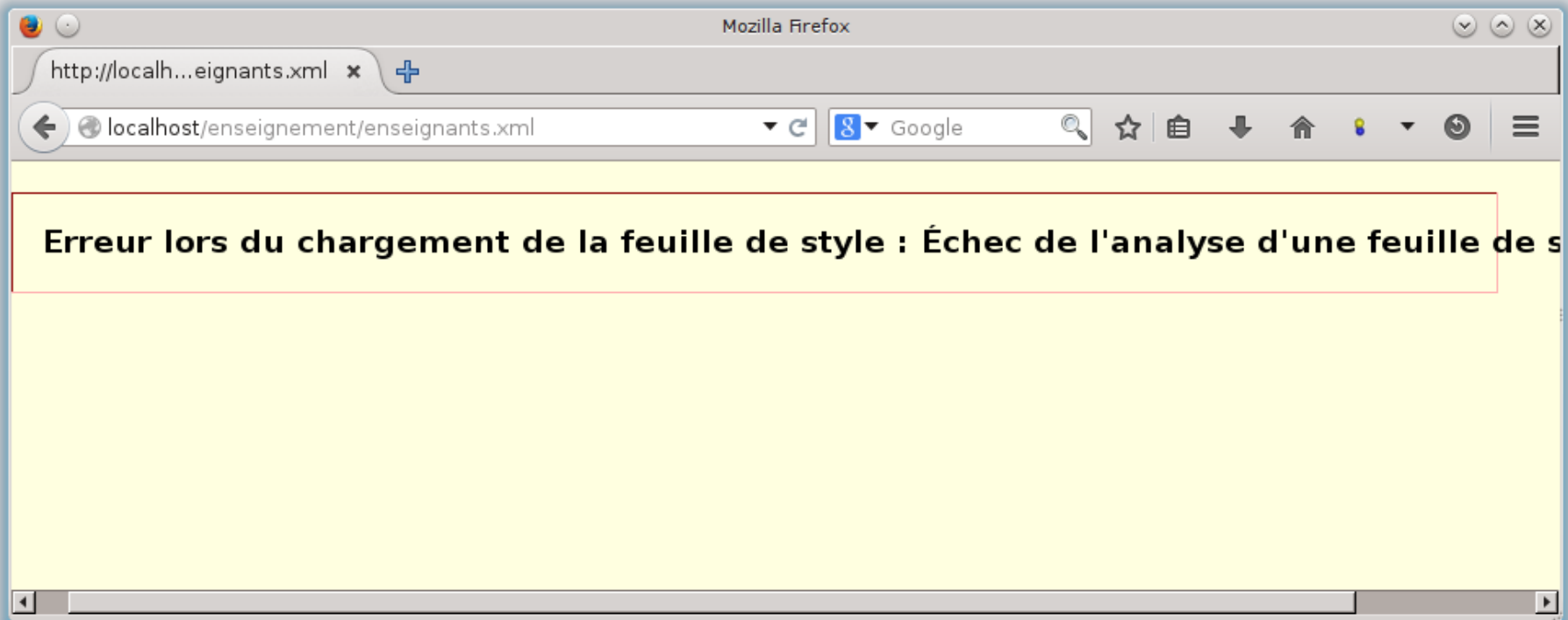
- Généralement, c'est le navigateur (côté client) qui effectue, « à la volée », la transformation.
- Si on tape Ctrl+U (affichage du code source), on voit le XML (pas le code HTML engendré automatiquement).
- Pour examiner de près le code HTML engendré :
`xsltproc enseignement.xsl enseignants.xml`

La transformation XSLT

- Elle reste généralement cachée (transparente pour l'utilisateur) ;
- par contre, elle n'est *pas du tout* faite pour être facilement déboguée !

79

La transformation XSLT



80

La transformation XSLT

- Elle reste généralement cachée (transparente pour l'utilisateur) ;
- par contre, elle n'est *pas du tout* fait pour être facilement déboguée !
- débrouillez-vous !
- (→ on a intérêt à travailler par petites étapes)

81

XSLT : comment ça marche ?

- Ajouter une ligne dans le document XML :

```
<?xml version="1.0"?>
<?xml-stylesheet type="text/xsl" href="enseignement.xsl"?>
<enseignants>
  <enseignant sexe="m">
    <prenom>Pascal</prenom>
    <nom>Vaillant</nom>
    <note>Ça c'est <b>moi</b> !</note>
  </enseignant>
  <enseignant sexe="f">
    <prenom>Sophie</prenom>
    <nom>Toulouse</nom>
    <note>Super-Sophie</note>
  </enseignant>
  ...
</enseignants>
```

82

XSLT : comment ça marche ?

- Ajouter une ligne dans le document XML :

```
<?xml version="1.0"?>
```

```
<?xml-stylesheet type="text/xsl" href="enseignement.xsl"?>
```

```
<enseignants>
```

```
  <enseignant sexe="m">
```

```
    <prenom>Pascal</prenom>
```

```
    <nom>Vaillant</nom>
```

```
    <note>Ça c'est <b>moi</b> !</note>
```

```
  </enseignant>
```

```
  <enseignant sexe="f">
```

```
    <prenom>Sophie</prenom>
```

```
    <nom>Toulouse</nom>
```

```
    <note>Super-Sophie</note>
```

```
  </enseignant>
```

```
  ...
```

```
</enseignants>
```

XML processing instruction

83

XSLT : comment ça marche ?

- Ajouter une ligne dans le document XML :

```
<?xml version="1.0"?>
```

```
<?xml-stylesheet type="text/xsl" href="enseignement.xsl"?>
```

```
<enseignants>
```

```
  <enseignant sexe="m">
```

```
    <prenom>Pascal</prenom>
```

```
    <nom>Vaillant</nom>
```

```
    <note>Ça c'est <b>moi</b> !</note>
```

```
  </enseignant>
```

```
  <enseignant sexe="f">
```

```
    <prenom>Sophie</prenom>
```

```
    <nom>Toulouse</nom>
```

```
    <note>Super-Sophie</note>
```

```
  </enseignant>
```

```
  ...
```

```
</enseignants>
```

XML processing instruction

C'est quelque chose de spécial (pas un élément XML ordinaire), comme tout ce qui est marqué par
<? ... ?>

84

XSLT : comment ça marche ?

- Ajouter une ligne dans le document XML :

```
<?xml version="1.0"?>
```

```
<?xml-stylesheet type="text/xsl" href="enseignement.xsl"?>
```

```
<enseignants>
```

```
  <enseignant sexe="m">
```

```
    <prenom>Pascal</prenom>
```

```
    <nom>Vaillant</nom>
```

```
    <note>Ça c'est <b>moi</b> !</note>
```

```
  </enseignant>
```

```
  <enseignant sexe="f">
```

```
    <prenom>Sophie</prenom>
```

```
    <nom>Toulouse</nom>
```

```
    <note>Super-Sophie</note>
```

```
  </enseignant>
```

```
  ...
```

```
</enseignants>
```

XML processing instruction

Celle-ci ("xml-stylesheet") sert à indiquer l'URL de la feuille XSLT qu'on conseille d'utiliser pour visualiser ce code XML

85

XSLT : comment ça marche ?

- Ensuite c'est le travail du navigateur !

86

www.univ-paris13.fr

www.iut-bobigny.univ-paris13.fr

Villetaneuse • Bobigny • Saint-Denis • Saint-Denis - La Plaine • Argenteuil

www.limics.smbh.univ-paris13.fr/Portail

Jerome.nobecourt@gmail.com

vaillant@univ-paris13.fr



L'Université Paris 13 est
membre fondateur de :



XSLT : comment ça marche ?

- Ensuite c'est le travail du navigateur !
- Si on veut examiner le HTML engendré :
`xsltproc enseignement.xsl enseignants.xml`

87

XSLT : comment ça marche ?

- Ensuite c'est le travail du navigateur !
- Si on veut examiner le HTML engendré :
`xsltproc enseignement.xsl enseignants.xml`
- Si on veut vérifier le code source XML :
`xmllint --schema enseignement.xsd enseignants.xml`

88

XSLT : comment ça marche ?

- Ensuite c'est le travail du navigateur !
- Si on veut examiner le HTML engendré :
`xsltproc enseignement.xsl enseignants.xml`
- Si on veut vérifier le code source XML :
`xmllint --schema enseignement.xsd enseignants.xml`
- Adresse d'un excellent tutoriel :

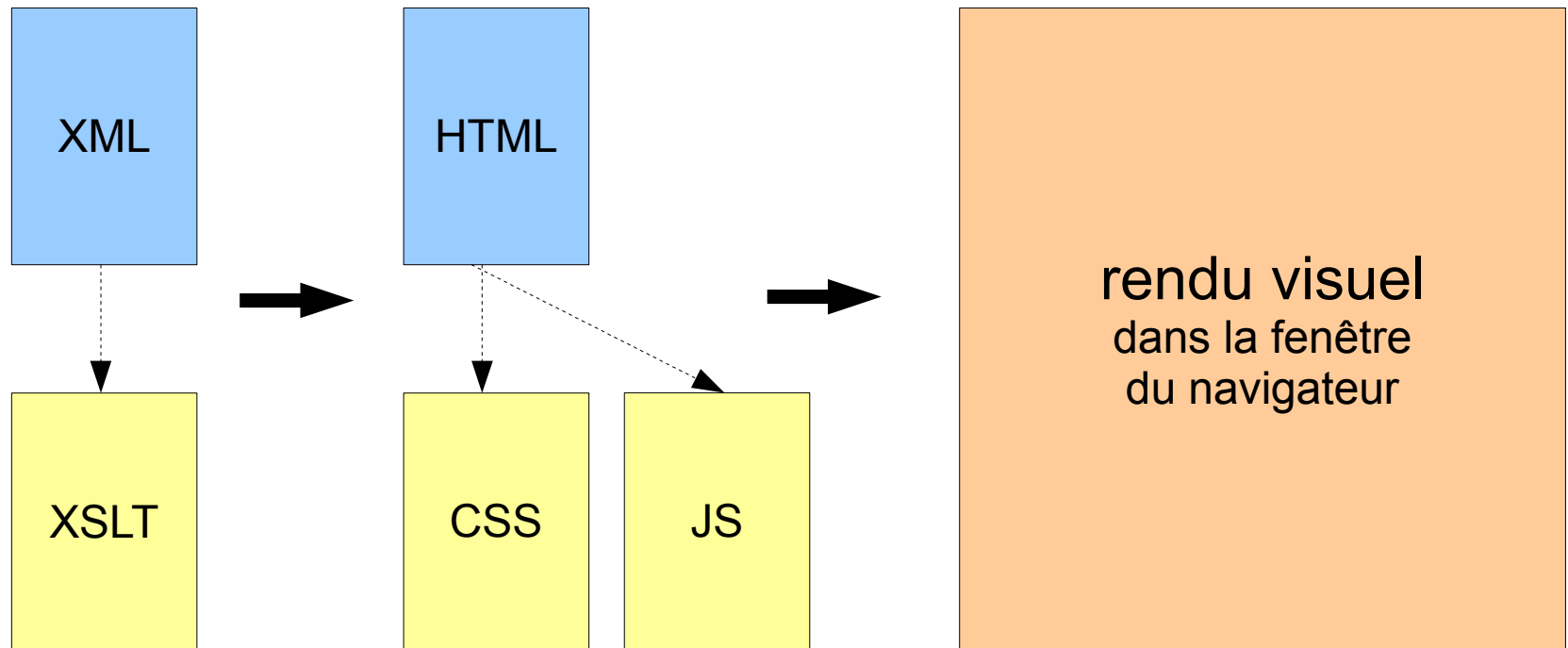
<http://www.w3schools.com/xsl/>

89

XSLT et CSS

- On peut utiliser des CSS avec XSLT
- Pour cela, il faut prévoir d'inclure le lien vers la feuille de style CSS (la *vraie* feuille de style) dans le document HTML engendré automatiquement par XSLT
- Pas de référence au CSS dans le XML !!!
- Dans le XML, on ne fait référence qu'au XSLT

XSLT et CSS



91

XSLT et CSS

- On met ça dans le document XML :

```
<?xml version="1.0"?>
```

```
<?xml-stylesheet type="text/xsl" href="enseignement.xsl"?>
```

```
<enseignants>
```

```
  <enseignant sexe="m">
```

```
    <prenom>Pascal</prenom>
```

```
    <nom>Vaillant</nom>
```

```
    <note>Ça c'est <b>moi</b> !</note>
```

```
  </enseignant>
```

```
  <enseignant sexe="f">
```

```
    <prenom>Sophie</prenom>
```

```
    <nom>Toulouse</nom>
```

```
    <note>Super-Sophie</note>
```

```
  </enseignant>
```

```
  ...
```

```
</enseignants>
```

XML processing instruction

92

XSLT et CSS

- Puis on met ça dans la feuille XSLT :

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0" xmlns:xsl="http://www.w3.org/1999/XSL/
Transform">
<xsl:output method="html"/>
<xsl:template match="/">
  <html lang="fr" dir="ltr">
    <head>
      <meta http-equiv="Content-Type" content="text/html;
charset=utf-8"/>
      <title>Liste des enseignants</title>
      <link rel="stylesheet" type="text/css" href="enseignement.css"
media="screen"></link>
    </head>
    <body><xsl:apply-templates/></body>
  </html>
</xsl:template>
```

*Lien vers le CSS, dans l'en-tête
du document HTML engendré*

93

XSLT et JavaScript, AJAX, etc.

- Ce qui vient d'être expliqué pour CSS est valable également pour Javascript :
- on peut prévoir d'inclure, dans le HTML automatiquement engendré par XSLT, des liens vers des fichiers contenant des fonctions JS, ou des fonctions JS en ligne ;
- ces fonctions JS peuvent à leur tour contenir des *XMLHttpRequest* pour déclencher des événements serveur ...
- bref tout est possible à condition de penser à l'inclure dans le XSL

94

XSLT : penser récursif

- Dans XSLT, les blocs de base constituent des règles de transformation qui disent : « quand je reconnais ceci dans le flot d'entrée, je mets cela dans le flot de sortie »

95

XSLT : penser récursif

- Dans XSLT, les blocs de base constituent des règles de transformation qui disent : « quand je reconnais ceci dans le flot d'entrée, je mets cela dans le flot de sortie »

```
<xsl:template match="/">  
  <html><head/><body>Bonjour</body></html>  
</xsl:template>
```

96

XSLT : penser récursif

- Dans XSLT, les blocs de base constituent des règles de transformation qui disent : « quand je reconnais ceci dans le flot d'entrée, je mets cela dans le flot de sortie »

```
<xsl:template match="/">  
  <html><head/><body>Bonjour</body></html>  
</xsl:template>
```

- On les appelle des « templates » (règles₉₇ génériques)

XSLT : penser récursif

- Dans l'exemple présenté ici, dès qu'on trouve l'élément racine ("`/`"), on le remplace (en sortie) par `<html><head/><body>Bonjour</body></html>`.
- Les nœuds inférieurs de l'arbre ne sont pas pris en compte.
- Pour les prendre en compte, il existe une instruction de « propagation » : *apply templates*.

98

XSLT : penser récursif

- Quand on a affaire à un document XML, soit on connaît d'avance exactement sa structure à tous les niveaux de profondeur, et l'on peut programmer très exactement la mise en forme finale, en HTML, de chaque information (on y accède par des expressions Xpath)
- ... soit on ne la connaît pas, et on se borne à indiquer des règles du type : un nœud de type <note> doit être transformé en un nœud de type <p>, et puis on plonge à l'intérieur et on voit comment traiter son contenu⁹⁹

XSLT : penser récursif

- La première méthode est parfaite pour des documents XML du type « export de table de bases de données »
- la seconde méthode est celle qu'on utilise généralement pour les documents XML qui sont de « vrais » documents (ordonnés, semi-structurés, récursifs, à profondeur variable ...)

... bref des données qui sortent d'un cerveau humain.

100

XSLT : instructions de base

`<xsl:template match="X">`

- Règle « template » : dit, de manière générique, ce qu'il faut faire quand on rencontre un nœud de type X.
- ce que l'on met sous cet élément, et qui n'est pas une instruction XSL, est recopié en sortie.

101

XSLT : instructions de base

`<xsl:apply-templates/>`

- Exemple basique d'utilisation avec `<xsl:template>` :

```
<xsl:template match="/">
  <html>
    <head>Joli document</head>
    <body><xsl:apply-templates/></body>
  </html>
</xsl:template>
```

102

XSLT : instructions de base

<xsl:apply-templates/>

- Parcourir le sous-arbre enraciné dans le nœud élément actuellement en cours d'examen, en appliquant éventuellement toutes les règles qui peuvent s'appliquer au fur et à mesure qu'on les rencontre.
- Pour chaque nœud du sous-arbre pour lequel aucune règle particulière ne s'applique : on le recopie tel quel.

103

XSLT : instructions de base

```
<xsl:value-of select="X"/>
```

- Copier ici le contenu de X, X étant une expression XPath.
- Exemple :

```
Sexe = <i><xsl:value-of select="./@sexe"/></i>
```

104

XSLT : instructions de base

```
<xsl:copy-of select="X" />
```

- Copier ici l'intégralité du nœud désigné par X, X étant une expression Xpath (avec son sous-arbre).
- Utile si le type de nœud fait partie de la grammaire d'entrée comme de la grammaire de sortie.

105

XSLT : instructions de contrôle

- On peut aussi (et on a besoin, parfois) d'utiliser des expressions XSLT permettant d'implémenter des instructions de contrôle.
- C'est utile même si XSLT reste fondamentalement récursif et basé sur des « templates ».

106

XSLT : instructions de contrôle

```
<xsl:for-each select="X"/>
```

- Ce qui est indiqué à l'intérieur de cette balise est répété dans le flux de sortie autant de fois que l'on trouve un élément répondant à la spécification XPath X.
- Utile pour appliquer une certaine transformation sur un ensemble de nœuds fils, par exemple.

107

XSLT : instructions de contrôle

```
<xsl:choose>  
  <xsl:when test="X"/>...</xsl:when>  
  <xsl:otherwise>...</xsl:otherwise>  
</xsl:choose>
```

- Instruction de branchement multiple : X étant un test sur une expression XPath, cette instruction permet de définir quoi envoyer dans le flux de sortie dans un certain nombre de cas prédéfinis ...
- et de prévoir un cas par défaut (¹⁰⁸<xsl:otherwise>)

XSLT : variables

```
<xsl:variable name="nb_fils">  
  <xsl:value-of select="count(child::nodes())"/>  
</xsl:variable>
```

- Permet de définir une variable.
- On peut la réutiliser ensuite par : \$nb_fils

109

XSLT : fonctions annexes

```
<xsl:template name="fonction">
```

```
...
```

```
</xsl:template>
```

- Permet de définir un bloc d'instructions dont on se ressert souvent.
- On peut l'appeler ensuite par :

```
<xsl:call-template name="fonction">
```

```
...
```

```
</xsl:call-template>
```

110

XSLT : manipulation du DOM

- Dans certains cas, on ne peut pas se contenter de créer des nœuds en les insérant directement dans le XSLT (par exemple parce qu'on ne connaît pas à l'avance le type d'élément qu'on veut créer, ou la valeur d'un attribut, et que ceux-ci doivent dépendre d'un calcul fait au moment de l'application de la feuille XSLT).

⇒ Il existe des instructions pour créer dynamiquement des éléments et des attributs.

111

XSLT : manipulation du DOM

```
<xsl:element name="X"/>
```

- Sert à créer un élément de type X (X pouvant être une expression calculée dynamiquement au moment de l'application de la feuille de transformations).

112

XSLT : manipulation du DOM

`<xsl:attribute name="X"/>`

- Sert à créer un attribut de type X (X pouvant être une expression calculée dynamiquement au moment de l'application de la feuille de transformations).
- Cet attribut est automatiquement rattaché au nœud élément correspondant à la position en cours dans l'arbre de sortie.
- La valeur de l'attribut est précisée dans le contenu.

Rappel: règle d'or de l'informatique

- Soyez tolérants dans ce que vous acceptez, et exigeant dans ce que vous produisez.
- Ça veut dire : (1) ne faites aucun postulat sur le format des données d'entrées ; ne comptez pas sur le respect de la moindre norme ; vérifiez le type et le format des données, essayez de traiter au mieux celles que vous pouvez récupérer, et écartez celles dont vous ne pouvez rien faire (c'est une règle de libéralité, mais aussi une règle de méfiance contre les injections).

114

Rappel: règle d'or de l'informatique

- Soyez tolérants dans ce que vous acceptez, et exigeant dans ce que vous produisez.
- Ça veut dire : (2) respectez les normes et les standards ; utilisez, quand vous avez le choix, les fonctions les plus génériques ; indiquez autant de paramètres que possible autant de fois que possible (par exemple l'encodage de caractères) ; testez sur plusieurs systèmes.

115